

Esercizio 1

Data la seguente porzione di programma rispondere alle domande corrispondenti motivando le risposte date:

```
int main() {

    int* matricola = new int[6]{...inserisci la tua matricola...};
    int* tmp = new int [1];
    int& r = matricola[0];
    int* p = matricola;

    // 1. Le seguenti istruzioni sono corrette? Motivare

    for (int i=0; i < 6; i++)    tmp[i]+= matricola[i];

    // 2. L'istruzione seguente è corretta? Se sì, cosa stampa?
    cout << r << endl;

    // 3. Le seguenti istruzioni sono corrette? Se sì, che cosa stampano?
    if (*p == r) cout << "Yes" << endl;
    else cout << "No" << endl;

    if (p == &r) cout << "Yes" << endl;
    else cout << "No" << endl;

    // 4. Scrivere sul foglio le operazioni necessarie per deallocare la
    memoria dinamica utilizzata
    ...
    return 0;

}
```

Esercizio 2

Gli studenti di una classe spesso chiedono ricevimento al docente per chiarire i propri dubbi. Si richiede di implementare una classe `CalendarioRicevimenti`, che tenga traccia delle richieste di ricevimento da parte degli studenti al docente di un corso. La classe implementata deve fornire almeno le seguenti funzionalità:

- 1) Ricevere una richiesta di ricevimento, tramite un metodo

```
bool chiediRicevimento(string studente, string giorno, int ora)
```

Il metodo deve dapprima controllare che il docente sia libero in quel giorno/ora, e in caso memorizzare l'appuntamento con lo studente e restituire `true`. Se occupato, il metodo restituisce `false` e non modifica niente.

- 2) Cancellare tutti i ricevimenti del docente in un determinato giorno, tramite il metodo

```
void libera(string& giorno)
```

- 3) Determinare se il docente è libero in un giorno/ora, tramite un metodo privato

```
bool isLibero(string& giorno, int ora)
```

- 4) La ridefinizione dell'operator== per confrontare due `CalendarioRicevimenti`.
Due calendari sono uguali se e solo se coincidono esattamente gli slot (giorno, ora) occupati, indipendentemente dagli studenti che li occupano.

Si può assumere che gli input siano sempre corretti, che il giorno sia formato da stringhe del tipo "DD/MM/YYYY", e che l'ora sia formata da numeri interi compresi nell'intervallo [8,19].

PICCOLO BONUS (Assegnato solo se gli altri metodi sono corretti). Ridefinizione dell'operator[] il quale, ricevendo in input un `giorno` (string), restituisca un `vector` contenente gli studenti che quel giorno sono impegnati con il docente.

Criteri che influenzeranno la valutazione dell'esercizio sono anche: la scelta delle strutture dati utilizzate per rappresentare il problema, l'uso opportuno della parola chiave `const` (le firme dei metodi vanno dunque riviste) e l'efficienza dei metodi implementati.

Esercizio 3

Dati in input un grafo orientato `g`, implementato tramite la classe `Grafo`, e due suoi nodi `s` e `t`, scrivere una funzione che restituisca `true` se e solo se esistono contemporaneamente:

- 1) un cammino da `s` a `t` e
- 2) un cammino da `t` ad `s`.

Non è necessario che il cammino sia quello di lunghezza minima. Non è necessario restituire il cammino né stamparlo in output. Per ottenere il punteggio pieno, la funzione che calcola il cammino deve essere ricorsiva.

La classe `Grafo` dispone dei seguenti metodi pubblici (dove `g` è un'istanza della classe `Grafo`):

- `g.n()` che restituisce il numero di nodi del Grafo
- `g.m()` che restituisce il numero di archi del Grafo
- `g(i, j)` che restituisce `true` se esiste un arco dal nodo `i` al nodo `j` e `false` altrimenti

Esercizio 4

Dato un `Grafo` non orientato `g` di `n` nodi, e un intero positivo `k`, scrivere una funzione che costruisca un vettore che associ ad ogni nodo un valore compreso tra **0** ed **n-1** tale che:

- 1) non ci siano due nodi adiacenti con valore uguale in modulo 3. In altri termini, per ogni arco (i, j) , si abbia che $v[i] \% 3 \neq v[j] \% 3$.
- 2) Per ogni nodo `x` del Grafo, non deve essere possibile assegnare $v[x] = x$
- 3) La somma dei valori assegnati ai nodi deve essere esattamente `k`

Se quest'assegnazione non esiste, il programma dovrà stampare **IMPOSSIBILE**, se esiste, l'assegnazione dovrà essere stampata in output. Per l'interfaccia pubblica della classe `Grafo` si veda l'esercizio 3.